PROJECT PROPOSAL

Modern Data Engineering Pipeline

End-to-end Data Warehouse for Superstore Sales Data

DuckDB | dbt | Airflow | Docker

PREPARED BY

Ahmed Ali Hasaballah Zaki

DATE

October 1, 2026

Table of Contents

1. Executive Summary . . . 3

Key outcomes . . . 3

2. Project Background & Objectives . . . 3

2.1 Business problem . . . 3

2.2 Objectives . . . 3

2.3 Scope . . . 3

3. Source Data: Superstore Dataset . . . 4

4. Solution Architecture . . . 4

5. Technology Stack & Key Advantages . . . 5

5.1 DuckDB - the analytical engine . . . 5

5.2 dbt - transformation, testing and documentation . . . 5

5.3 Apache Airflow - orchestration . . . 5

5.4 Docker - reproducible environment . . . 5

6. Pipeline Design . . . 6

6.1 ODS load (Python + DuckDB) . . . 6

6.2 Staging layer (dbt view) . . . 6

6.3 Marts layer (dbt dimensional models) . . . 6

7. Dimensional Model: Star Schema . . . 7

7.1 Grain of the fact table . . . 7

7.2 Dimensions . . . 8

7.3 Questions the model answers . . . 8

8. Surrogate Key Strategy . . . 8

9. Orchestration with Airflow . . . 9

Reference DAG definition . . . 10

10. Code Review & Recommended Fixes . . . 10

11. Data Quality & Testing Plan . . . 11

11.1 Simple generic tests (schema.yml) . . . 11

11.2 Simple singular tests (SQL queries) . . . 13

11.3 Running the tests . . . 14

12. Docker Deployment Plan . . . 15

Proposed project structure . . . 15

13. Deliverables & Timeline . . . 15

14. Risks & Mitigations . . . 16

15. Conclusion & Next Steps . . . 16

Appendix A. Corrected dbt Models . . . 17

1. Executive Summary

This proposal presents an end-to-end, modern data engineering pipeline that converts the raw **Superstore** sales file into a trusted, analytics-ready **star-schema data warehouse**. The solution is built entirely on open-source tools, each with a clear responsibility:

Tool	Role in the pipeline
DuckDB	Embedded analytical database that stores the ODS, staging and data-mart layers.
dbt	SQL transformation framework: staging cleansing, dimensional modelling, tests and documentation.
Apache Airflow	Orchestrator that runs and monitors the pipeline as a dependency-aware DAG.
Docker	Packaging layer that makes the whole stack portable and reproducible on any machine.

The pipeline is already implemented and has been validated: the Airflow DAG `ods_staging_marts` completed with all three tasks (`load_to_ods`, `dbt_staging`, `dbt_marts`) in **success** state, and the final fact table `main_dwh.fact_orders` was loaded with **9,994 rows**.

Key outcomes

- A single, repeatable command-line / UI-triggered run that goes from CSV to star schema.
- A clean separation of layers: **ODS** (raw copy), **Staging** (cleansed), **Marts** (modelled).
- A star schema with one fact table and six conformed dimensions, ready for BI tools.
- Deterministic **surrogate keys** generated with `dbt_utils.generate_surrogate_key` so keys stay stable across rebuilds.
- A foundation that scales: the same code can later target a cloud warehouse with only an adapter change.

2. Project Background & Objectives

2.1 Business problem

The Superstore dataset is delivered as one flat CSV file with 21 columns. Customer, product, location, shipping and financial attributes are mixed together in every row, dates are stored as text, and text values contain inconsistent spacing and casing. Analysts must repeat cleaning logic in every report, results are hard to reproduce, and there is no automation or quality control.

2.2 Objectives

- Ingest the raw file automatically into an Operational Data Store (ODS).
- Standardise names, data types, dates and text values in a staging layer.
- Model the data as a **Kimball-style star schema** for fast, intuitive analytics.
- Automate execution, retries and monitoring with Airflow.
- Package everything in Docker so that any teammate can run the project identically.
- Add automated data-quality tests and generated documentation with dbt.

2.3 Scope

In scope	Out of scope (future phases)
CSV ingestion, ODS, staging, dimensional marts, Airflow DAG, Docker packaging, dbt tests and docs.	Cloud deployment, real-time/streaming ingestion, dashboards and machine-learning models.

3. Source Data: Superstore Dataset

The source is `data/raw/Superstore.csv`. Each row represents one product line within a customer order. Dates use the `m/d/Y` text format. The 21 columns fall into five groups:

Group	Columns	Comment
Order	Row ID, Order ID, Order Date, Ship Date, Ship Mode	Row ID is unique per line; Order ID repeats across lines.
Customer	Customer ID, Customer Name, Segment	Segment = consumer / corporate / home office.
Location	Country, City, State, Postal Code, Region	Geographic hierarchy of the delivery address.
Product	Product ID, Category, Sub-Category, Product Name	Two-level product hierarchy.
Measures	Sales, Quantity, Discount, Profit	Numeric facts used for analysis.

4. Solution Architecture

Data moves left to right through three layers. Airflow triggers each step in order, dbt performs the SQL transformations inside DuckDB, and Docker hosts the runtime.

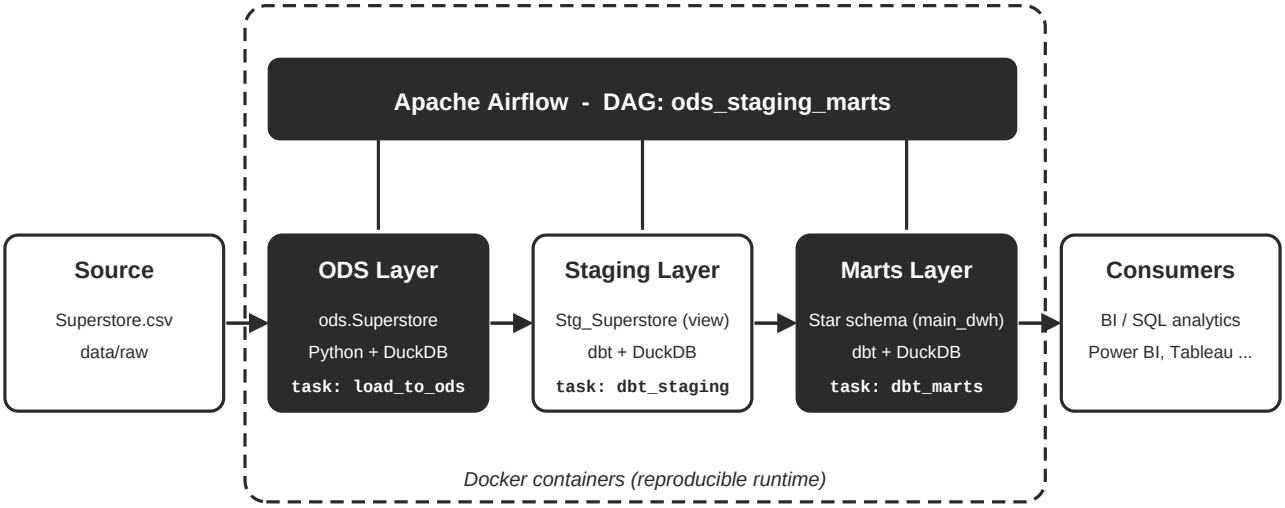


Figure 1 - High-level pipeline architecture

Layer	Object	Built by	Purpose
Raw	Superstore.csv	-	Immutable source file.
ODS	ods.Superstore (table)	Python + DuckDB	Untouched copy of the source inside the database; full reload on each run.
Staging	Stg_Superstore (view)	dbt	Rename, cast, trim and standardise every column.
Marts	dim_* and fact_orders (schema main_dwh)	dbt	Business-friendly star schema for analytics.

5. Technology Stack & Key Advantages

5.1 DuckDB - the analytical engine

- **Zero administration:** runs in-process, no server to install; the whole warehouse is one file (`dev.duckdb`).
- **Fast analytics:** columnar, vectorised execution handles millions of rows on a laptop.
- **Rich SQL:** built-in functions such as `strptime`, `generate_series` and `unnest` simplify date and cleansing logic.
- **Reads CSV/Parquet directly** and integrates natively with pandas and Python.
- **Free and portable:** no licence cost; models can later move to MotherDuck, Snowflake or BigQuery through dbt adapters.

5.2 dbt - transformation, testing and documentation

- **SQL-first and version-controlled:** every transformation is a reviewable `SELECT` statement in Git.
- **Automatic dependency graph:** `ref()` and `source()` build the lineage, so models always run in the right order.
- **Reusable logic:** Jinja macros and packages such as `dbt_utils` (surrogate keys, tests) remove copy-paste code.
- **Built-in testing:** `unique`, `not_null`, `relationships` and custom tests protect data quality.
- **Living documentation:** `dbt docs generate` produces column descriptions and a visual lineage graph.
- **Flexible materialisation:** choose view or table per model with one config line.

5.3 Apache Airflow - orchestration

- **Pipelines as code:** DAGs are Python files, versioned and peer-reviewed like any software.
- **Dependency control:** `load_to_ods >> dbt_staging >> dbt_marts` guarantees the correct order and stops downstream tasks if one fails.
- **Scheduling and backfills:** run hourly, daily or on demand, and re-run past periods easily.
- **Retries and alerting:** automatic retry policies plus email/Slack notifications on failure.
- **Observability:** a web UI with graph view, Gantt chart and per-task logs (see Section 9).
- **Extensible:** hundreds of provider packages for databases, clouds and APIs when the project grows.

5.4 Docker - reproducible environment

- **Same environment everywhere:** identical Python, dbt, DuckDB and Airflow versions on every machine.
- **One-command start-up:** `docker compose up` replaces long manual installation guides.
- **Isolation:** dependencies never conflict with other projects on the host.
- **Easy onboarding and CI/CD:** the same image runs on a laptop, a build server or a cloud cluster.

> **Why this combination?** DuckDB removes infrastructure cost, dbt gives engineering discipline to SQL, Airflow adds reliable automation, and Docker makes the result portable. Together they form a lightweight yet production-style stack that mirrors what larger teams run in the cloud.

6. Pipeline Design

6.1 ODS load (Python + DuckDB)

A Python script resolves the project paths, reads the CSV with pandas, registers the DataFrame in DuckDB and runs `CREATE OR REPLACE TABLE ods.Superstore AS SELECT * FROM csv_data`. The schema is created if missing, the load is **idempotent** (safe to re-run) and a row count is logged for monitoring.

6.2 Staging layer (dbt view)

The model `Stg_Superstore` reads from `source('ods','Superstore')` and applies consistent cleansing rules:

Transformation	Example
Rename to snake_case	`"Row ID"` becomes `row_id`, `"Sub-Category"` becomes `sub_category`
Cast to proper types	`cast(Sales as decimal(18,2))`, `cast(Quantity as int)`, `cast(Discount as decimal(5,2))`
Parse text dates	`STRPTIME("Order Date", '%m/%d/%Y')::DATE`
Trim and lower-case categories	`lower(trim("Ship Mode"))`, segment, region, category, sub_category
Trim free-text fields	customer_name, country, city, state, product_name

6.3 Marts layer (dbt dimensional models)

Six dimension models and one fact model are built from the staging view. Dimensions hold descriptive attributes with one row per member; `fact_orders` joins staging to each dimension to replace business values with surrogate keys and keeps only the numeric measures.

7. Dimensional Model: Star Schema

The warehouse follows Kimball dimensional modelling. A central fact table is surrounded by six dimensions, giving analysts simple joins and fast aggregation.

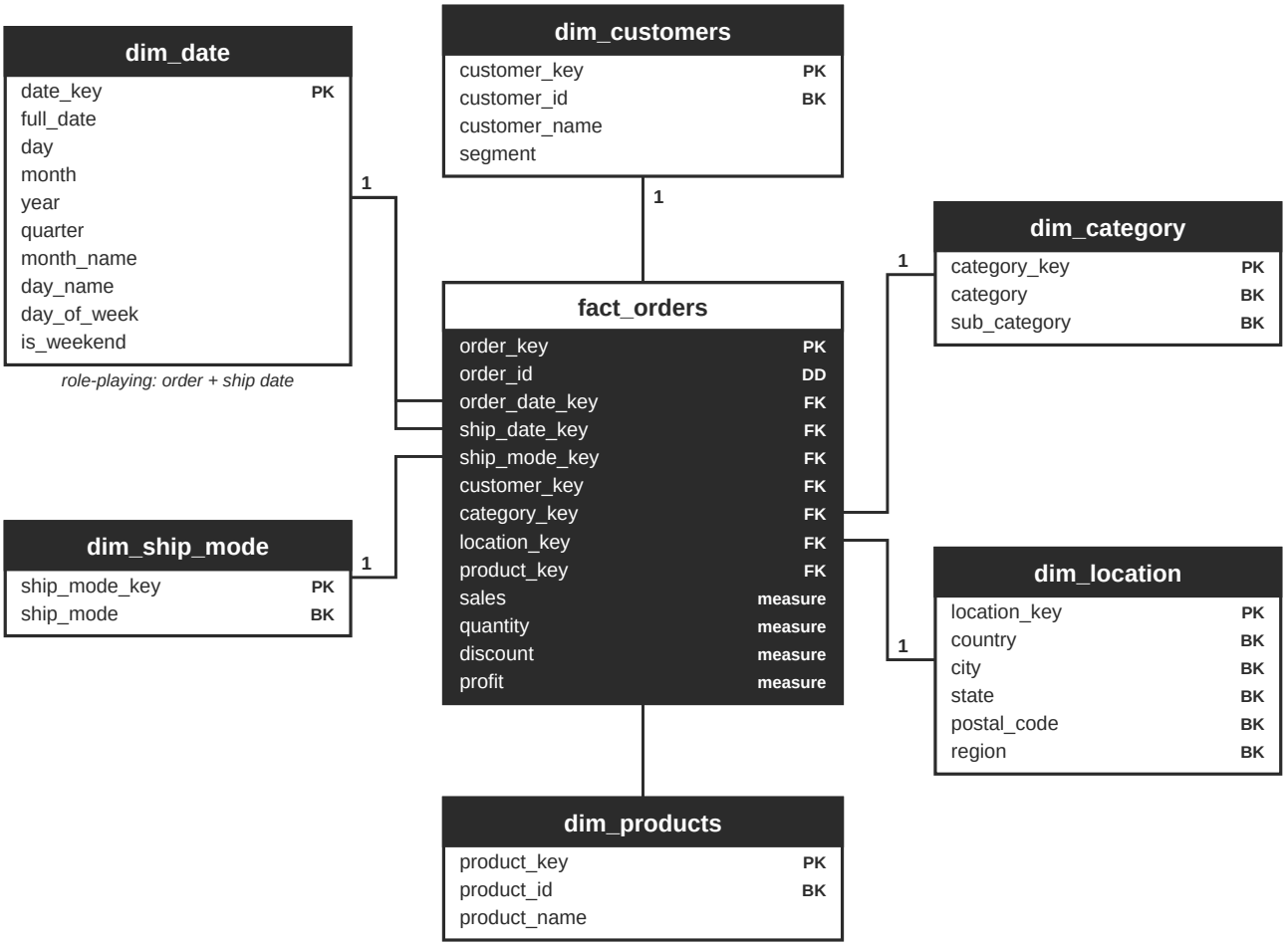


Figure 2 - Star schema of the Superstore data warehouse

7.1 Grain of the fact table

> **Grain of fact_orders:** one row per **order line item** - i.e. one product sold within one customer order, identified by the source `Row ID`. The loaded table contains **9,994** such rows.

Element	Columns	Notes
Surrogate key	order_key	Hash of row_id (see Section 8).
Degenerate dimension	order_id	Order number kept in the fact; no separate dimension needed.
Foreign keys	order_date_key, ship_date_key, ship_mode_key, customer_key, category_key, location_key, product_key	Date dimension is a **role-playing** dimension (order date and ship date).
Additive measures	sales, quantity, profit	Can be summed across any dimension.
Non-additive measure	discount	A rate (0-1): analyse as an average or weighted by sales, never sum.

7.2 Dimensions

Dimension	Business key	Attributes	Surrogate key
dim_date	full_date	day, month, year, quarter, month_name, day_name, day_of_week, is_weekend	date_key (YYYYMMDD smart key)
dim_customers	customer_id	customer_name, segment	customer_key
dim_products	product_id	product_name	product_key
dim_category	category + sub_category	category, sub_category	category_key
dim_location	country, city, state, postal_code, region	Full delivery geography	location_key
dim_ship_mode	ship_mode	ship_mode	ship_mode_key

dim_date is generated, not extracted: it creates one row for every calendar day between the minimum and maximum order date, so reports show continuous time series even for days without sales.

7.3 Questions the model answers

- Sales and profit by category, sub-category, region, state or city.
- Monthly, quarterly and yearly trends; weekday vs. weekend behaviour.
- Most and least profitable customers, segments and products.
- Impact of discount levels and ship modes on profit and delivery time (ship date vs. order date).

8. Surrogate Key Strategy

A surrogate key is a meaningless identifier that links the fact table to a dimension. The current code uses `ROW_NUMBER()` in several models. This is risky: the number assigned to a member depends on the sort order and on the data currently present, so keys can change after new data arrives or after a full rebuild, silently breaking joins and downstream reports.

The proposal is to use **deterministic hashed keys** everywhere through `dbt_utils.generate_surrogate_key` (already used in dim_customers and dim_products). The same business key always produces the same hash, independent of load order, row count or rebuild.

Model	Current key line	Proposed key line
dim_category	`ROW_NUMBER() OVER (ORDER BY category, sub_category)`	`{{ dbt_utils.generate_surrogate_key(['category','sub_category']) }}`
dim_ship_mode	`199 + ROW_NUMBER() OVER (ORDER BY ship_mode)`	`{{ dbt_utils.generate_surrogate_key(['ship_mode']) }}`
dim_location	`(not shown - assumed ROW_NUMBER)`	`{{ dbt_utils.generate_surrogate_key(['country','city','state','postal_code','region']) }}`
fact_orders	`row_number() over (order by s.row_id)`	`{{ dbt_utils.generate_surrogate_key(['row_id']) }}`
dim_customers	already hashed	`{{ dbt_utils.generate_surrogate_key(['customer_id']) }}`
dim_products	already hashed	`{{ dbt_utils.generate_surrogate_key(['product_id']) }}`
dim_date	`yyyymmdd integer`	keep: a smart date key is standard practice and already deterministic

Setup: add the package to `packages.yml` and run `dbt deps` once.

```
packages:
  - package: dbt-labs/dbt_utils
    version: [">=1.1.0", "<2.0.0"]
```

Important: because the fact table joins to the dimensions on business columns and copies the dimension's key, no join logic changes. Every dimension key and the fact key are simply replaced by the hashed version, so the foreign keys in `fact_orders` stay consistent automatically. The full corrected code is in Appendix A.

9. Orchestration with Airflow

The DAG `ods_staging_marts` chains three BashOperator tasks. Each task starts only if the previous one succeeded, so a failed load can never produce partial marts.

Task	Operator	What it runs	Result
load_to_ods	BashOperator	Python loader: CSV to ods.Superstore	success
dbt_staging	BashOperator	dbt run for the staging model	success
dbt_marts	BashOperator	dbt run for dimensions and fact	success

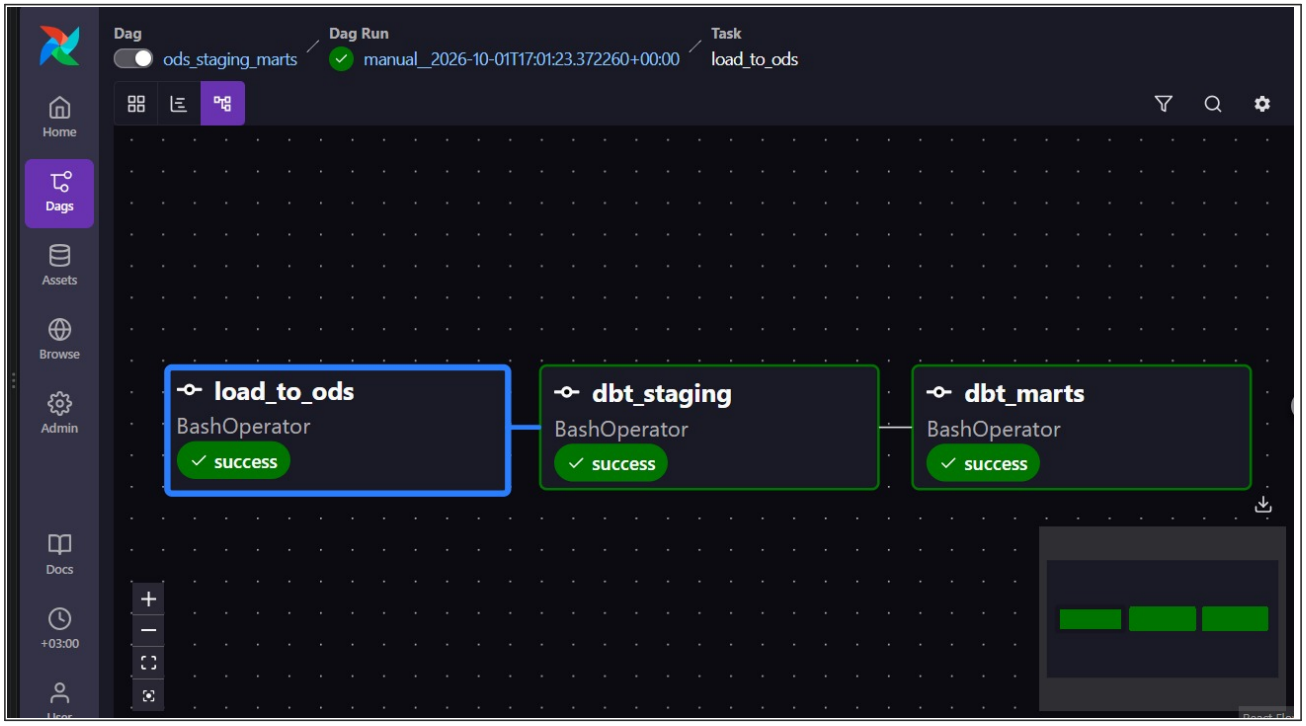


Figure 3 - Airflow graph view: DAG run manual__2026-10-01T17:01:23 finished with all tasks in success state

Task log - load_to_ods. The first task runs the Python loader inside the Airflow container (`/usr/local/airflow/dbt_project/scripts/load_to_ods.py`). The log shows **9,994 rows** loaded into `ods.Superstore`, the follow-up row-count check, and exit code 0.

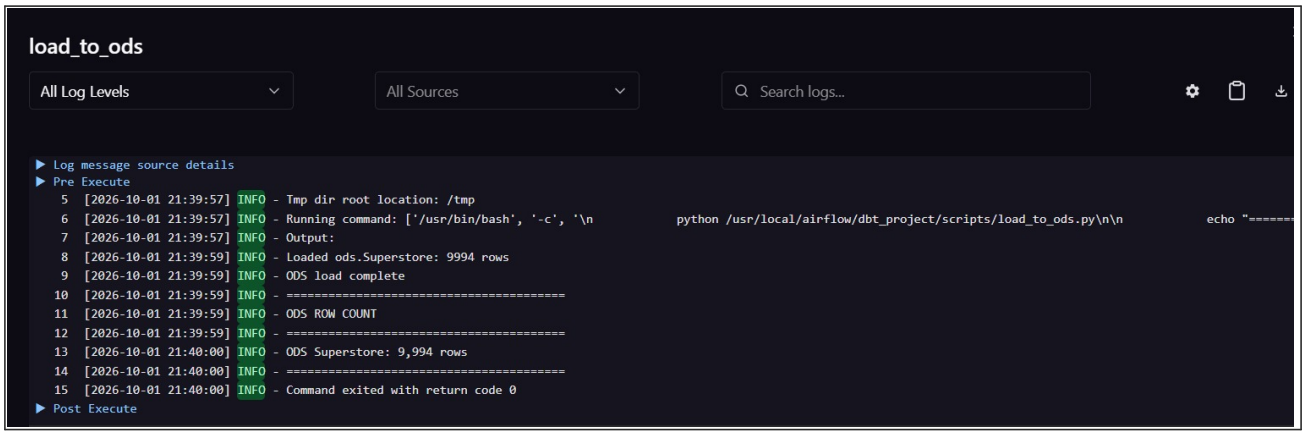


Figure 4 - Airflow log of task load_to_ods: ods.Superstore loaded with 9,994 rows

Task log - dbt_marts. The final step confirms that `main_dwh.fact_orders` also contains **9,994 rows**, so no row was lost or duplicated between the ODS and the fact table, and the command exited with return code 0.

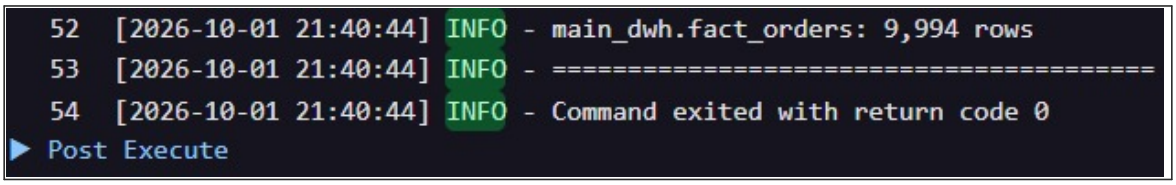


Figure 5 - Task log excerpt: fact_orders row count and successful exit code

Reference DAG definition

Illustrative code; the loader path is taken from your task log, the dbt commands and selectors should be adjusted to your project.

```
from datetime import datetime
from airflow import DAG
from airflow.providers.standard.operators.bash import BashOperator

with DAG("ods_staging_marts", start_date=datetime(2026, 1, 1),
         schedule=None, catchup=False, tags=["superstore"]) as dag:

    load_to_ods = BashOperator(
        task_id="load_to_ods",
        bash_command="python /usr/local/airflow/dbt_project/scripts/load_to_ods.py")

    dbt_staging = BashOperator(
        task_id="dbt_staging",
        bash_command="cd /usr/local/airflow/dbt_project && dbt run --select staging")

    dbt_marts = BashOperator(
        task_id="dbt_marts",
        bash_command="cd /usr/local/airflow/dbt_project && dbt run --select marts")

    load_to_ods >> dbt_staging >> dbt_marts
```

10. Code Review & Recommended Fixes

During review of the current code base the following improvements were identified. They are low-risk and are already applied in Appendix A.

#	Finding	Recommendation
1	Python lines (`con.unregister(...)`, `con.close()`, `print(...)`) appear at the end of `Stg_Superstore` and `dim_date` SQL.	Remove them from dbt models; they belong only in the Python loader.
2	Trailing comma after the last column (`Profit,`) before `from source`.	Remove the comma.

#	Finding	Recommendation
3	Column typo `dicount` in staging and fact.	Rename to `discount` in both models.
4	Inconsistent alias `Profit` (capital P).	Use lowercase `profit` for consistent naming.
5	`dim_customers` selects customer_name and segment but groups only by customer_id.	Group by all selected columns (or use SELECT DISTINCT) and add a unique test on customer_key.
6	Dimensions are materialised as views, so hashes are recomputed on every query.	Materialise dimensions as tables (marts layer) for speed and stable joins.
7	ROW_NUMBER keys in dim_category, dim_ship_mode and fact_orders.	Use hashed surrogate keys (Section 8).
8	Mixed model naming (`Stg_Superstore` vs `dim_*`).	Adopt lowercase `stg_superstore` convention in a later refactor.
9	Loader goes through pandas before DuckDB.	Optionally use `read_csv_auto(path)` directly in DuckDB (faster, lower memory) and add a `loaded_at` audit column.
10	Fact joins use LEFT JOIN; a missing dimension match would give NULL keys silently.	Add `relationships` tests and a row-count reconciliation check.

11. Data Quality & Testing Plan

dbt tests run after the marts are built and fail the Airflow task when data does not meet expectations. The table below summarises the plan; Sections 11.1 to 11.3 provide ready-to-use test code.

Test type	Applied to	Purpose
unique + not_null	All surrogate keys (customer_key, product_key, category_key, ship_mode_key, location_key, date_key, order_key)	Guarantee one row per member and no missing keys.
relationships	Every foreign key in fact_orders to its dimension	Detect orphan facts.
accepted_values	ship_mode, segment, region, category	Catch unexpected category spellings.
Row-count reconciliation	fact_orders vs Stg_Superstore (expected 9,994)	Detect row loss or join fan-out.
Sum reconciliation	sum(sales) and sum(profit) fact vs staging	Prove no financial value is lost.
Business rules	ship_date >= order_date; discount between 0 and 1; quantity > 0	Validate logical consistency.

11.1 Simple generic tests (schema.yml)

Generic tests are declared in YAML, with no SQL to write. Create `models/staging/schema.yml` and `models/marts/schema.yml`. (dbt older than 1.8: write `tests:` instead of `data_tests:`.)

Staging tests

```
version: 2

models:
  - name: Stg_Superstore
    columns:
      - name: row_id
        data_tests: [unique, not_null]
      - name: order_id
        data_tests: [not_null]
      - name: ship_mode
        data_tests:
          - accepted_values:
              values: ['first class', 'second class', 'standard class', 'same day']
      - name: segment
        data_tests:
          - accepted_values:
              values: ['consumer', 'corporate', 'home office']
      - name: region
        data_tests:
          - accepted_values:
              values: ['central', 'east', 'south', 'west']
      - name: category
        data_tests:
          - accepted_values:
              values: ['furniture', 'office supplies', 'technology']
```

Marts tests - unique/not_null on every surrogate key, and a relationships test on every fact foreign key:

```
version: 2

models:
  - name: dim_customers
    columns:
      - name: customer_key
        data_tests: [unique, not_null]
  - name: dim_products
    columns:
      - name: product_key
        data_tests: [unique, not_null]
  - name: dim_category
    columns:
      - name: category_key
        data_tests: [unique, not_null]
  - name: dim_ship_mode
    columns:
      - name: ship_mode_key
        data_tests: [unique, not_null]
  - name: dim_location
    columns:
      - name: location_key
        data_tests: [unique, not_null]
  - name: dim_date
    columns:
      - name: date_key
        data_tests: [unique, not_null]

  - name: fact_orders
    columns:
      - name: order_key
        data_tests: [unique, not_null]
      - name: order_date_key
        data_tests:
          - not_null
          - relationships: {to: ref('dim_date'), field: date_key}
      - name: ship_date_key
        data_tests:
          - not_null
          - relationships: {to: ref('dim_date'), field: date_key}
      - name: ship_mode_key
        data_tests:
          - not_null
          - relationships: {to: ref('dim_ship_mode'), field: ship_mode_key}
      - name: customer_key
        data_tests:
          - not_null
          - relationships: {to: ref('dim_customers'), field: customer_key}
      - name: category_key
        data_tests:
          - not_null
          - relationships: {to: ref('dim_category'), field: category_key}
      - name: location_key
        data_tests:
          - not_null
          - relationships: {to: ref('dim_location'), field: location_key}
      - name: product_key
        data_tests:
          - not_null
          - relationships: {to: ref('dim_products'), field: product_key}
```

> **Tip:** adjust the `accepted_values` lists if your data contains other spellings. A failing `not_null` test on a fact foreign key means a LEFT JOIN found no matching dimension row.

11.2 Simple singular tests (SQL queries)

A singular test is a plain SQL query saved in the `tests/` folder. The test **passes when the query returns zero rows**, so each query selects only the bad records.

tests/assert_fact_rowcount_matches_staging.sql - no rows lost or duplicated by the joins:

```
select *
from (
    select
        (select count(*) from {{ ref('fact_orders') }})    as fact_rows,
        (select count(*) from {{ ref('Stg_Superstore') }}) as staging_rows
)
where fact_rows <> staging_rows
```

tests/assert_sales_and_profit_match.sql - totals in the fact equal totals in staging:

```
select *
from (
    select
        (select sum(sales)  from {{ ref('fact_orders') }})    as fact_sales,
        (select sum(sales)  from {{ ref('Stg_Superstore') }}) as stg_sales,
        (select sum(profit) from {{ ref('fact_orders') }})    as fact_profit,
        (select sum(profit) from {{ ref('Stg_Superstore') }}) as stg_profit
)
where fact_sales <> stg_sales
   or fact_profit <> stg_profit
```

tests/assert_ship_date_not_before_order_date.sql:

```
select row_id, order_date, ship_date
from {{ ref('Stg_Superstore') }}
where ship_date < order_date
```

tests/assert_discount_and_quantity_valid.sql:

```
select row_id, discount, quantity
from {{ ref('Stg_Superstore') }}
where discount < 0
   or discount > 1
   or quantity <= 0
```

11.3 Running the tests

Command	What it does
`dbt deps`	Installs dbt_utils (once).
`dbt test`	Runs all generic and singular tests.
`dbt test --select fact_orders`	Tests one model only.
`dbt test --select test_type:singular`	Runs only the SQL tests in the tests/ folder.
`dbt test --select test_type:generic`	Runs only the YAML tests.
`dbt build`	Runs models and their tests together, in dependency order.

To make Airflow enforce quality, add a fourth task after the marts. If any test fails, the task fails and the run is marked as failed:

```
dbt_test = BashOperator(
    task_id="dbt_test",
    bash_command="cd /usr/local/airflow/dbt_project && dbt test")

load_to_ods >> dbt_staging >> dbt_marts >> dbt_test
```

12. Docker Deployment Plan

A single compose file starts all Airflow components (API server, scheduler, DAG processor) and a metadata database, with the project folder mounted so DAGs, scripts and the dbt project are shared. The DuckDB file lives on a mounted volume so data survives container restarts.

Proposed project structure

```
dbt_project/          (mounted in Airflow at /usr/local/airflow/dbt_project)
|-- scripts/
|   `-- load_to_ods.py             # CSV -> ods.Superstore
|-- data/raw/Superstore.csv        # source file
|-- models/
|   |-- staging/Stg_Superstore.sql, schema.yml
|   `-- marts/dim_*.sql, fact_orders.sql, schema.yml
|-- tests/                         # singular SQL tests
|-- dbt_project.yml
|-- profiles.yml                   # DuckDB connection (dev.duckdb)
|-- packages.yml                   # dbt_utils
`-- dev.duckdb                     # warehouse file (volume)

dags/ods_staging_marts.py          # Airflow DAG
Dockerfile                         # Airflow + dbt-duckdb + duckdb + pandas
docker-compose.yml
```

- **Start:** `docker compose up -d` then open the Airflow UI and trigger `ods_staging_marts`.
- **Concurrency note:** DuckDB allows one writer at a time. The tasks are sequential, so this is safe; avoid opening the file in another tool for writing while the DAG runs.
- **Secrets and config:** keep paths and credentials in environment variables, never in the image.

13. Deliverables & Timeline

Phase	Deliverable	Indicative effort	Status
1. Ingestion	Python loader and ods.Superstore	0.5 day	Completed
2. Staging	Stg_Superstore cleansing view	0.5 day	Completed
3. Dimensional model	6 dimensions and fact_orders	1-2 days	Completed; surrogate-key refactor proposed
4. Orchestration	Airflow DAG ods_staging_marts	0.5 day	Completed (manual run succeeded)
5. Quality and docs	dbt tests, dbt docs, README	1 day	Planned
6. Containerisation	Dockerfile and docker-compose.yml	1 day	To finalise
7. BI and scheduling	Dashboard connection, daily schedule	1-2 days	Planned

14. Risks & Mitigations

Risk	Impact	Mitigation
Source CSV format or encoding changes	Load fails or columns shift	Schema checks in the loader; ODS keeps an untouched copy; dbt source tests.
Duplicate business keys (e.g. one product_id with two names)	Fact rows multiply on join	unique tests on dimensions and row-count reconciliation of the fact.
DuckDB single-writer limit	Lock errors if accessed concurrently	Sequential DAG; read-only BI connections or export to Parquet.
Growth beyond one machine	Slower runs	Switch dbt adapter to a cloud warehouse; models remain unchanged.
Environment drift between developers	It works on my machine	Docker image with pinned versions.

15. Conclusion & Next Steps

The proposed pipeline delivers a complete, automated path from raw Superstore data to a tested star schema, using a modern stack that is free, fast and easy to reproduce. The core build is already working end to end. The recommended next steps are:

- Apply the hashed surrogate keys and code fixes listed in Sections 8 and 10 (Appendix A).
- Add dbt tests and generate project documentation.
- Finalise the Docker packaging and set a daily Airflow schedule.
- Connect a BI tool (Power BI, Tableau or Metabase) to the marts for dashboards.

Appendix A. Corrected dbt Models

All models below use deterministic surrogate keys. `dim_location` was not included in the supplied code, so it is reconstructed from the join conditions in `fact_orders` - please compare with your original file.

A.1 Stg_Superstore.sql

```
{{ config(materialized='view') }}

with source as (
    select * from {{ source('ods','Superstore') }}
)

select
    cast("Row ID" as int)                       as row_id,
    "Order ID"                                  as order_id,
    strptime("Order Date", '%m/%d/%Y')::date    as order_date,
    strptime("Ship Date",  '%m/%d/%Y')::date    as ship_date,
    lower(trim("Ship Mode"))                    as ship_mode,
    "Customer ID"                               as customer_id,
    trim("Customer Name")                       as customer_name,
    lower(trim("Segment"))                      as segment,
    trim("Country")                             as country,
    trim("City")                                as city,
    trim("State")                               as state,
    cast("Postal Code" as int)                  as postal_code,
    lower(trim("Region"))                       as region,
    "Product ID"                                as product_id,
    trim("Product Name")                        as product_name,
    lower(trim("Category"))                     as category,
    lower(trim("Sub-Category"))                 as sub_category,
    cast("Sales" as decimal(18,2))              as sales,
    cast("Quantity" as int)                     as quantity,
    cast("Discount" as decimal(5,2))            as discount,
    cast("Profit" as decimal(18,2))             as profit
from source
```

A.2 dim_customers.sql

```
{{ config(materialized='table') }}

-- Deterministic surrogate key: the same customer_id always gives the same hash,
-- unlike ROW_NUMBER() which changes when data or sort order changes.
select
    {{ dbt_utils.generate_surrogate_key(['customer_id']) }} as customer_key,
    customer_id,
    customer_name,
    segment
from {{ ref('Stg_Superstore') }}
group by customer_id, customer_name, segment
```

A.3 dim_products.sql

```
{{ config(materialized='table') }}

select
    {{ dbt_utils.generate_surrogate_key(['product_id']) }} as product_key,
    product_id,
    product_name
from {{ ref('Stg_Superstore') }}
group by product_id, product_name
```

A.4 dim_category.sql

```
{{ config(materialized='table') }}

select
    {{ dbt_utils.generate_surrogate_key(['category', 'sub_category']) }} as category_key,
    category,
    sub_category
from {{ ref('Stg_Superstore') }}
group by category, sub_category
```

A.5 dim_ship_mode.sql

```
{{ config(materialized='table') }}

select
    {{ dbt_utils.generate_surrogate_key(['ship_mode']) }} as ship_mode_key,
    ship_mode
from {{ ref('Stg_Superstore') }}
group by ship_mode
```

A.6 dim_location.sql (reconstructed)

```
{{ config(materialized='table') }}

select
    {{ dbt_utils.generate_surrogate_key(
        ['country', 'city', 'state', 'postal_code', 'region']) }} as location_key,
    country,
    city,
    state,
    postal_code,
    region
from {{ ref('Stg_Superstore') }}
group by country, city, state, postal_code, region
```

A.7 dim_date.sql

```
{{ config(materialized='table') }}

with date_range as (
    select min(order_date) as min_date, max(order_date) as max_date
    from {{ ref('Stg_Superstore') }}
),

dates as (
    -- generate_series returns a list of dates; unnest turns each element into a row
    select unnest(generate_series(min_date, max_date, interval '1 day'))::date as full_date
    from date_range
)

select
    cast(strftime(full_date, '%Y%m%d') as int)      as date_key,
    full_date,
    extract(day from full_date)::int                as day,
    extract(month from full_date)::int              as month,
    extract(year from full_date)::int               as year,
    'Q' || extract(quarter from full_date)::int     as quarter,
    strftime(full_date, '%B')                       as month_name,
    strftime(full_date, '%A')                       as day_name,
    extract(isodow from full_date)::int             as day_of_week,
    extract(isodow from full_date) in (6, 7)        as is_weekend
from dates
```

A.8 fact_orders.sql

```
{{ config(materialized='table') }}

select
    {{ dbt_utils.generate_surrogate_key(['s.row_id']) }} as order_key,
    s.order_id,
    dt.date_key       as order_date_key,
    ds.date_key       as ship_date_key,
    sm.ship_mode_key,
    cst.customer_key,
    cat.category_key,
    loc.location_key,
    prd.product_key,
    s.sales,
    s.quantity,
    s.discount,
    s.profit
from {{ ref('Stg_Superstore') }} s
left join {{ ref('dim_date') }}      dt  on s.order_date = dt.full_date
left join {{ ref('dim_date') }}      ds  on s.ship_date  = ds.full_date
left join {{ ref('dim_ship_mode') }} sm  on s.ship_mode  = sm.ship_mode
left join {{ ref('dim_customers') }} cst on s.customer_id = cst.customer_id
left join {{ ref('dim_category') }}  cat on s.category = cat.category
                                        and s.sub_category = cat.sub_category
left join {{ ref('dim_location') }}  loc on s.country = loc.country
                                        and s.city = loc.city
                                        and s.state = loc.state
                                        and s.postal_code = loc.postal_code
                                        and s.region = loc.region
left join {{ ref('dim_products') }}  prd on s.product_id = prd.product_id
```